

# Algoritmi e strutture dati

## Analisi di algoritmi

### Complessità algoritmi vs Complessità problemi

Alberto Montresor

Università di Trento

2026/08/21

This work is licensed under a Creative Commons  
Attribution-ShareAlike 4.0 International License.



# Introduzione

Obiettivo: **riflettere su complessità di problemi/algoritmi**

- In alcuni casi, si può migliorare quanto si ritiene "normale"
- In altri casi, è impossibile fare di meglio
- Qual è il rapporto fra un problema computazionale e l'algoritmo?

Back to basics!

- Somme
- Moltiplicazioni

# Moltiplicare numeri complessi

## Moltiplicazione numeri complessi

- $(a + bi)(c + di) = [ac - bd] + [ad + bc]i$
- Input:  $a, b, c, d$
- Output:  $a \times c - b \times d, a \times d + b \times c$

## Domande

Considerate un modello di calcolo dove la moltiplicazione costa 1, le addizioni/sottrazioni costano 0.01,

- Quanto costa l'algoritmo dettato dalla definizione?
- Potete fare meglio di così?
- Qual è il ruolo del modello di calcolo?

# Moltiplicare numeri complessi

## Moltiplicazione numeri complessi

- $(a + bi)(c + di) = [ac - bd] + [ad + bc]i$
- Input:  $a, b, c, d$
- Output:  $a \times c - b \times d, a \times d + b \times c$

## Si può fare meglio di così!

$$p_{ac} = a \times c$$

$$p_{bd} = b \times d$$

$$s = (a + b) \times (c + d) = ac + ad + bc + bd$$

$$\text{Output: } [p_{ac} - p_{bd}] + [s - p_{ac} - p_{bd}]i$$

**Tre moltiplicazioni invece di quattro:** 25% in meno nel modello scelto

# Sommare numeri binari

## Algoritmo elementare della somma

Procede da destra verso sinistra (dal LSB al MSB), sommando due bit e l'eventuale riporto, con costo totale  $\Theta(n)$ .

$$\begin{array}{cccccccc}
 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\
 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \\
 \hline
 +1 & +1 & & & +1 & +1 & & \\
 \hline
 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1
 \end{array}$$

## Domanda

Si può fare meglio di così?

# Limite superiore alla complessità di un problema

## Notazione $\mathcal{O}(f(n))$ – Limite superiore

Fissato un modello di calcolo, un problema ha una **limitazione superiore**  $\mathcal{O}(f(n))$  se esiste almeno un algoritmo corretto che lo risolve in tempo  $\mathcal{O}(f(n))$ .

## Limite superiore della somma di numeri binari

Il problema della somma di numeri binari ha una limitazione superiore  $\mathcal{O}(n)$ .

# Limite inferiore alla complessità di un problema

## Notazione $\Omega(f(n))$ – Limite inferiore

Fissato un modello di calcolo, un problema ha una **limitazione inferiore**  $\Omega(f(n))$  se ogni algoritmo corretto che lo risolve richiede tempo  $\Omega(f(n))$ .

## Limite inferiore della somma di numeri binari

Il problema della somma di numeri binari ha una limitazione inferiore  $\Omega(n)$ .

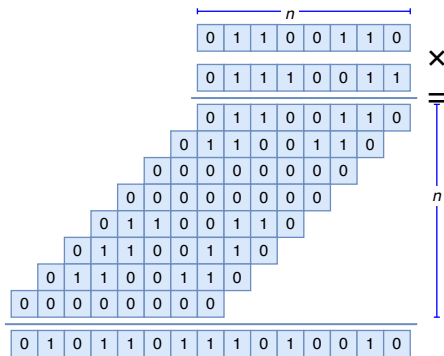
## Domanda

Riuscite a dimostrarlo?

# Moltiplicare numeri binari

## Algoritmo elementare del prodotto

L'algoritmo che abbiamo imparato a scuola genera un prodotto parziale per ogni bit di un operando e somma i prodotti parziali opportunamente traslati, con costo totale  $\Theta(n^2)$ .



# Algoritmi aritmetici

## Confronto della complessità computazionale

- Somma :  $T_{sum}(n) = \Theta(n)$
- Prodotto :  $T_{prod}(n) = \Theta(n^2)$

Si potrebbe concludere che...

- Il problema della moltiplicazione è inerentemente più costoso del problema dell'addizione
- Conferma la nostra esperienza

# Algoritmi aritmetici

## Attenzione!

- Abbiamo confrontato gli algoritmi, non i problemi!
- Sappiamo solo che l'*algoritmo elementare di somma* è più efficiente dell'*algoritmo elementare di prodotto*

## Confronto fra problemi

Per provare che il *problema* del prodotto è più costoso del *problema* della somma, dobbiamo provare che **non esiste** una soluzione in tempo lineare per il prodotto

## Un po' di storia

- Nel 1960, Kolmogorov enunciò in una conferenza che la moltiplicazione ha limite inferiore  $\Omega(n^2)$
- Una settimana dopo, un suo studente provò il contrario!

# Anticipazione

## Divide-et-impera

- **Divide**: dividi il problema in sottoproblemi di dimensioni inferiori
- **Impera**: risolvi i sottoproblemi in maniera ricorsiva
- **Combina**: unisci le soluzioni dei sottoproblemi in modo da ottenere la risposta del problema principale

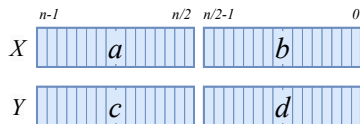
# Moltiplicare numeri binari

## Moltiplicazione divide-et-impera

$$X = a \cdot 2^{n/2} + b$$

$$Y = c \cdot 2^{n/2} + d$$

$$XY = ac \cdot 2^n + (ad + bc) \cdot 2^{n/2} + bd$$



Un'assunzione semplificativa:

- $n = 2^k$ , ovvero  $k = \log n$ ;
- tutti i sottovettori hanno dimensione pari a una potenza di 2.

Poi vedremo che queste assunzioni possono essere rimosse

# Quattro sottoprodotti: il codice

---

```
int Prodotto4(int X, int Y, int n)
```

---

```
if n == 1 then
```

```
    return X · Y
```

% Caso base: un solo bit

```
scomponi  $X = a 2^{n/2} + b$  e  $Y = c 2^{n/2} + d$ 
```

% Divide

```
int pac = Prodotto4(a, c, n/2)
```

% Impera

```
int pad = Prodotto4(a, d, n/2)
```

% Impera

```
int pbc = Prodotto4(b, c, n/2)
```

% Impera

```
int pbd = Prodotto4(b, d, n/2)
```

% Impera

```
int s = pad + pbc
```

```
return pac 2n + s 2n/2 + pbd
```

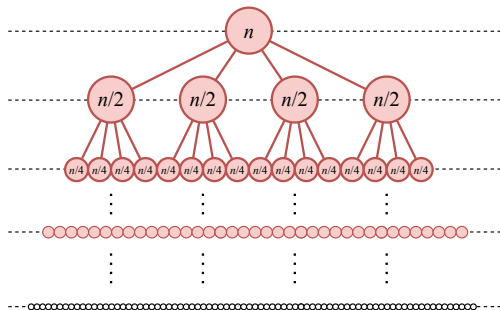
% Combina

---

$$T(n) = \begin{cases} d & n = 1, \\ 4T(n/2) + cn & n > 1. \end{cases}$$

Moltiplicare per  $2^t$  equivale a uno shift di  $t$  posizioni; somme, shift e ricomposizione hanno costo lineare.

# Analisi per livelli: quattro sottoprodotti



| Livello  | #            | Dim     | Costo                     |
|----------|--------------|---------|---------------------------|
| 0        | $4^0$        | $n/2^0$ | $c \cdot n/2^0 \cdot 4^0$ |
| 1        | $4^1$        | $n/2^1$ | $c \cdot n/2^1 \cdot 4^1$ |
| 2        | $4^2$        | $n/2^2$ | $c \cdot n/2^2 \cdot 4^2$ |
| $i$      | $4^i$        | $n/2^i$ | $c \cdot n/2^i \cdot 4^i$ |
| $\log n$ | $4^{\log n}$ | 1       | $d \cdot 4^{\log n}$      |

$$T(n) = \left( \sum_{i=0}^{k-1} c \cdot \frac{n}{2^i} \cdot 4^i \right) + d \cdot 4^k = \left( cn \cdot \sum_{i=0}^{k-1} 2^i \right) + d \cdot 4^k$$

# Analisi per livelli: quattro sottoprodotti

$$T(n) = \left( cn \cdot \sum_{i=0}^{k-1} 2^i \right) + d \cdot 4^k$$

## Costo dei livelli interni

$$\begin{aligned} cn \cdot \sum_{i=0}^{k-1} 2^i &= cn \cdot \frac{2^k - 1}{2 - 1} \\ &= cn(2^{\log n} - 1) \\ &= cn(n - 1) \end{aligned}$$

Somma geometrica finita

$$k = \log n$$

$$2^{\log n} = n$$

## Somma geometrica finita

$$\sum_{i=0}^t x^i = \frac{x^{t+1} - 1}{x - 1} \quad (x \neq 1)$$

# Analisi per livelli: quattro sottoprodotti

$$T(n) = \left( cn \cdot \sum_{i=0}^{k-1} 2^i \right) + d \cdot 4^k$$

## Costo delle foglie

$$d \cdot 4^k = d \cdot 4^{\log_2 n}$$

$$= d \cdot n^{\log_2 4}$$

$$= d \cdot n^2$$

$$k = \log_2 n$$

$$a^{\log_b n} = n^{\log_b a}$$

$$\log_2 4 = 2$$

## Complessità finale

$$T(n) = cn(n-1) + dn^2 = \Theta(n^2).$$

# Moltiplicare numeri binari

## Confronto della complessità computazionale

- Scolastico:  $T(n) = \Theta(n^2)$
- Quattro prodotti:  $T(n) = \Theta(n^2)$

## Tanto lavoro per nulla!

Non solo la complessità è uguale, ma le costanti moltiplicative sono più alte.

## Domanda

È possibile fare meglio di così?

Notate che la versione ricorsiva chiama se stessa 4 volte...

# Karatsuba: da quattro sottoprodotti a tre

---

```
int Karatsuba(int X, int Y, int n)
```

---

```
if  $n \leq 1$  then
```

```
    return  $X \cdot Y$ 
```

*% Lo stesso caso base*

```
scomponi  $X = a 2^{n/2} + b$  e  $Y = c 2^{n/2} + d$ 
```

```
int  $p_{ac} = \text{Karatsuba}(a, c, n/2)$ 
```

*%  $a \times c$*

```
int  $p_{bd} = \text{Karatsuba}(b, d, n/2)$ 
```

*%  $b \times d$*

```
int  $r = \text{Karatsuba}(a + b, c + d, n/2)$  %  $(a + b) \times (c + d) = ac + ad + bc + bd$ 
```

```
int  $s = r - p_{ac} - p_{bd}$ 
```

*%  $ad + bc$*

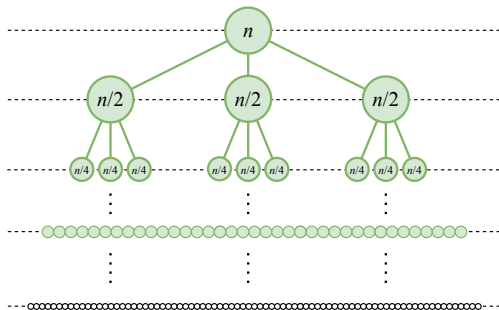
```
return  $p_{ac} 2^n + s 2^{n/2} + p_{bd}$ 
```

*% La stessa ricomposizione*

---

$$T(n) = \begin{cases} d & n = 1, \\ 3T(n/2) + cn & n > 1. \end{cases}$$

# Analisi per livelli: tre sottoprodotti



| Livello  | #            | Dim     | Costo                     |
|----------|--------------|---------|---------------------------|
| 0        | $3^0$        | $n/2^0$ | $c \cdot n/2^0 \cdot 3^0$ |
| 1        | $3^1$        | $n/2^1$ | $c \cdot n/2^1 \cdot 3^1$ |
| 2        | $3^2$        | $n/2^2$ | $c \cdot n/2^2 \cdot 3^2$ |
| $i$      | $3^i$        | $n/2^i$ | $c \cdot n/2^i \cdot 3^i$ |
| $\log n$ | $3^{\log n}$ | 1       | $d \cdot 3^{\log n}$      |

$$T(n) = \left( \sum_{i=0}^{k-1} c \cdot \frac{n}{2^i} \cdot 3^i \right) + d \cdot 3^k = \left( cn \cdot \sum_{i=0}^{k-1} \left(\frac{3}{2}\right)^i \right) + d \cdot 3^k$$

# Analisi per livelli: tre sottoprodotti

$$T(n) = \left( cn \cdot \sum_{i=0}^{k-1} (3/2)^i \right) + d \cdot 3^k$$

## Costo dei livelli interni

$$cn \sum_{i=0}^{k-1} (3/2)^i = cn \frac{(3/2)^k - 1}{3/2 - 1}$$

Somma geometrica finita

$$= 2cn \left( n^{\log_2(3/2)} - 1 \right)$$

$$k = \log_2 n$$

$$= 2c \left( n^{\log_2 3} - n \right)$$

$$\log_2 3/2 = (\log_2 3) - 1$$

## Somma geometrica finita

$$\sum_{i=0}^t x^i = \frac{x^{t+1} - 1}{x - 1} \quad (x \neq 1)$$

## Analisi per livelli: tre sottoprodotti

$$T(n) = \left( cn \cdot \sum_{i=0}^{k-1} (3/2)^i \right) + d \cdot 3^k$$

### Costo delle foglie

$$\begin{aligned} d \cdot 3^k &= d \cdot 3^{\log_2 n} \\ &= d \cdot n^{\log_2 3} \end{aligned}$$

$$\begin{aligned} k &= \log n \\ a^{\log_b n} &= n^{\log_b a} \end{aligned}$$

### Complessità finale

$$T(n) = 2c \left( n^{\log_2 3} - n \right) + dn^{\log_2 3} = \Theta(n^{\log_2 3}), \quad \log_2 3 \approx 1.585$$

# Moltiplicare numeri binari

## Confronto della complessità computazionale

- Scolastico:  $T_{prod}(n) = \Theta(n^2)$       Es.  $T_{prod}(10^6) = 10^{12}$
- Karatsuba:  $T_{kara}(n) = \Theta(n^{\log_2 3})$       Es.  $T_{kara}(10^6) \approx 3 \cdot 10^9$

## Conclusioni

- L'algoritmo "naif" non è sempre il migliore ...
- ... può esistere spazio di miglioramento ...
- ... a meno che non sia possibile dimostrare il contrario!

# Algoritmi vs problemi

## Complessità in tempo di un **algoritmo**

La quantità di tempo richiesta dall'algoritmo sugli input di dimensione  $n$ .

- possiamo analizzare caso peggiore, medio e migliore;
- otteniamo un limite sul costo di quella particolare soluzione.

## Complessità in tempo di un **problema computazionale**

- **Limitazione superiore**  $\mathcal{O}(f(n))$ : esiste un algoritmo corretto che risolve il problema in tempo  $\mathcal{O}(f(n))$ .
- **Limitazione inferiore**  $\Omega(g(n))$ : ogni algoritmo corretto richiede tempo  $\Omega(g(n))$ .
- Se i due ordini coincidono, l'algoritmo che raggiunge il limite inferiore è **asintoticamente ottimo**.

# Algoritmi e strutture dati

## Analisi di algoritmi Algoritmi di ordinamento

Alberto Montresor

Università di Trento

2026/08/21

This work is licensed under a Creative Commons  
Attribution-ShareAlike 4.0 International License.



# Introduzione

Obiettivo: **valutare gli algoritmi in base all'input**

- In alcuni casi, gli algoritmi si comportano diversamente a seconda delle caratteristiche dell'input
- Conoscere in anticipo tali caratteristiche permette di scegliere il miglior algoritmo in quella situazione
- Il problema dell'ordinamento è una buona palestra dove mostrare questi concetti

Algoritmi d'ordinamento

- Selection Sort
- Insertion Sort
- Merge Sort

# Tipologia di analisi

## Analisi del caso peggiore

- La più importante
- Il tempo di esecuzione nel caso peggiore è un **limite superiore** al tempo di esecuzione per qualsiasi input
- Per alcuni algoritmi, il caso peggiore si verifica molto spesso  
Es.: ricerca di dati non presenti in un database

## Analisi del caso medio

- Difficile in alcuni casi: cosa si intende per "medio"?
- Distribuzione uniforme

## Analisi del caso migliore

- Può avere senso se si hanno informazioni particolari sull'input

# Ordinamento

## Problema dell'ordinamento

- **Input:** Una sequenza  $A = a_1, a_2, \dots, a_n$  di  $n$  valori
- **Output:** Una sequenza  $B = b_1, b_2, \dots, b_n$  che sia una permutazione di  $A$  e tale per cui  $b_1 \leq b_2 \leq \dots \leq b_n$

Approccio "demente":

- Genero tutte le possibili permutazioni fino a quando non ne trovo una già ordinata

Approccio "naif":

- Cerco il minimo e lo metto in posizione corretta, riducendo il problema agli  $n - 1$  restanti valori.

# Selection Sort

---

**SelectionSort**(**int**[] *A*, **int** *n*)

---

**for** *i* = 0 **to** *n* - 2 **do**

**int** *pos* = **argmin**(*A*, *i*, *n* - 1)                      % Minimo della parte non ordinata  
    *A*[*i*], *A*[*pos*] = *A*[*pos*], *A*[*i*]                      % Estende il prefisso ordinato

**int** **argmin**(**int**[] *A*, **int** *start*, **int** *end*)

**int** *pos* = *start*                                      % Posizione del minimo parziale  
    **for** *j* = *start* + 1 **to** *end* **do**  
        **if** *A*[*j*] < *A*[*pos*] **then**  
            *pos* = *j*                                      % Nuovo minimo parziale  
    **return** *pos*

---

## Selection Sort: esecuzione

|         |   |   |   |   |   |   |   |
|---------|---|---|---|---|---|---|---|
| $i = 0$ | 7 | 4 | 2 | 1 | 8 | 3 | 5 |
| $i = 1$ | 1 | 4 | 2 | 7 | 8 | 3 | 5 |
| $i = 2$ | 1 | 2 | 4 | 7 | 8 | 3 | 5 |
| $i = 3$ | 1 | 2 | 3 | 7 | 8 | 4 | 5 |
| $i = 4$ | 1 | 2 | 3 | 4 | 8 | 7 | 5 |
| $i = 5$ | 1 | 2 | 3 | 4 | 5 | 7 | 8 |
| $i = 6$ | 1 | 2 | 3 | 4 | 5 | 7 | 8 |

## Selection Sort: costo

All'iterazione  $i$ , `argmin()` confronta l'elemento candidato con i successivi  $n - i - 1$  elementi:

$$\sum_{i=0}^{n-2} (n - i - 1) = \sum_{j=1}^{n-1} j = \frac{n(n-1)}{2} = \Theta(n^2).$$

Il costo non dipende dall'ordine iniziale

Selection Sort esamina comunque tutta la parte non ordinata:  
caso migliore = caso medio = caso peggiore =  $\Theta(n^2)$ .

# Insertion Sort

- Algoritmo efficiente per ordinare piccoli insiemi di elementi
- Si basa sul principio di ordinamento di una "mano" di carte da gioco (e.g. scala quaranta)

---

**InsertionSort**(**int**[] *A*, **int** *n*)

---

**for** *i* = 1 **to** *n* - 1 **do**

**int** *temp* = *A*[*i*]

        % Elemento da inserire

**int** *j* = *i*

        % Scorre il prefisso ordinato

**while** *j* > 0 **and** *A*[*j* - 1] > *temp* **do**

*A*[*j*] = *A*[*j* - 1]

        % Sposta a destra un elemento maggiore

*j* = *j* - 1

*A*[*j*] = *temp*

        % Inserisce nella posizione corretta

---

# Insertion Sort: esecuzione

|                | 0        | 1 | 2 | 3 | 4 | 5 | 6 | <i>temp</i> |
|----------------|----------|---|---|---|---|---|---|-------------|
|                | 7        | 4 | 2 | 1 | 8 | 3 | 5 |             |
| $i = 1, j = 1$ | 7        | 7 | 2 | 1 | 8 | 3 | 5 | 4           |
| $i = 1, j = 0$ | <u>4</u> | 7 | 2 | 1 | 8 | 3 | 5 | 4           |
| $i = 2, j = 2$ | 4        | 7 | 7 | 1 | 8 | 3 | 5 | 2           |
| $i = 2, j = 1$ | 4        | 4 | 7 | 1 | 8 | 3 | 5 | 2           |
| $i = 2, j = 0$ | <u>2</u> | 4 | 7 | 1 | 8 | 3 | 5 | 2           |

# Insertion Sort: esecuzione

|                | 0        | 1 | 2 | 3 | 4        | 5 | 6 | <i>temp</i> |
|----------------|----------|---|---|---|----------|---|---|-------------|
| $i = 3, j = 3$ | 2        | 4 | 7 | 7 | 8        | 3 | 5 | 1           |
| $i = 3, j = 2$ | 2        | 4 | 4 | 7 | 8        | 3 | 5 | 1           |
| $i = 3, j = 1$ | 2        | 2 | 4 | 7 | 8        | 3 | 5 | 1           |
| $i = 3, j = 0$ | <u>1</u> | 2 | 4 | 7 | 8        | 3 | 5 | 1           |
| $i = 4, j = 4$ | 1        | 2 | 4 | 7 | <u>8</u> | 3 | 5 | 8           |
| $i = 5, j = 5$ | 1        | 2 | 4 | 7 | 8        | 8 | 5 | 5           |

# Insertion Sort: esecuzione

|                | 0 | 1 | 2        | 3 | 4        | 5 | 6 | <i>temp</i> |
|----------------|---|---|----------|---|----------|---|---|-------------|
| $i = 5, j = 4$ | 1 | 2 | 4        | 7 | 7        | 8 | 5 | 3           |
| $i = 5, j = 3$ | 1 | 2 | 4        | 4 | 7        | 8 | 5 | 3           |
| $i = 5, j = 2$ | 1 | 2 | <u>3</u> | 4 | 7        | 8 | 5 | 3           |
| $i = 6, j = 6$ | 1 | 2 | 3        | 4 | 7        | 8 | 8 | 5           |
| $i = 6, j = 5$ | 1 | 2 | 3        | 4 | 7        | 7 | 8 | 5           |
| $i = 6, j = 4$ | 1 | 2 | 3        | 4 | <u>5</u> | 7 | 8 | 5           |

# Insertion Sort: correttezza e costo

## Invariante

All'inizio dell'iterazione  $i$ , il prefisso  $A[0 \dots i - 1]$  è ordinato. L'iterazione inserisce  $A[i]$  nella posizione corretta ed estende il prefisso ordinato fino ad  $A[0 \dots i]$ .

## Il costo dipende dalla disposizione dei dati

| Caso     | Struttura dell'input       | Costo         |
|----------|----------------------------|---------------|
| Migliore | già ordinato               | $\Theta(n)$   |
| Medio    | permutazioni equiprobabili | $\Theta(n^2)$ |
| Peggior  | ordinato al contrario      | $\Theta(n^2)$ |

# Merge Sort

## Divide et impera

Merge Sort è basato sulla tecnica **divide-et-impera** vista in precedenza

- **Divide**: Spezza virtualmente il vettore di  $n$  elementi in due sottovettori di  $n/2$  elementi
- **Impera**: Chiama Merge Sort ricorsivamente sui due sottovettori
- **Combina**: Unisci (**merge**) le due sequenze ordinate

## Idea

Si sfrutta il fatto che i due sottovettori sono già ordinati per ordinare più velocemente

# Merge

## Input

- Due porzioni adiacenti già ordinate:  
 $A[start \dots mid]$  e  $A[mid + 1 \dots end]$
- Un vettore di appoggio  $B$

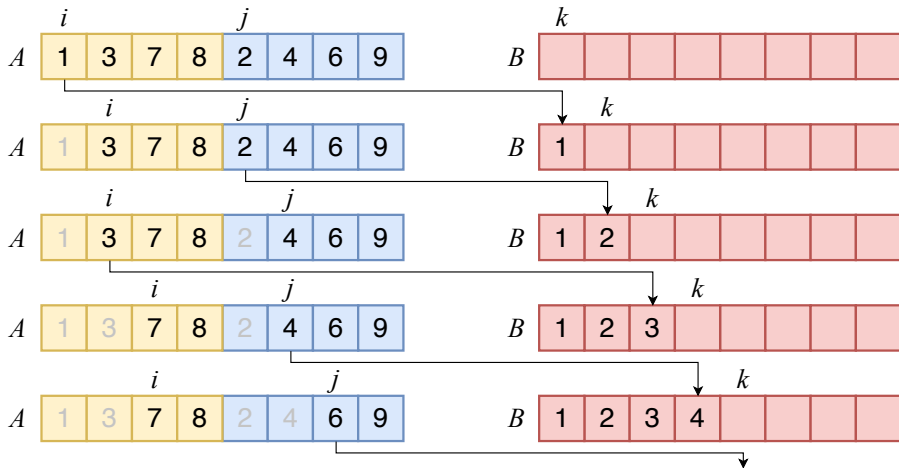
## Idea

Confrontiamo il primo elemento non ancora utilizzato delle due porzioni. Il minore è anche il minore fra tutti gli elementi rimanenti e può essere copiato nella prossima posizione libera di  $B$ .

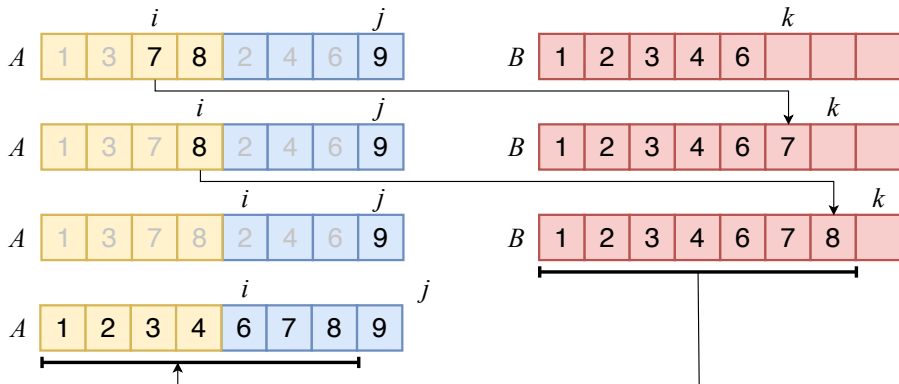
## Output

La porzione  $A[start \dots end]$  contiene tutti gli elementi delle due porzioni, in ordine non decrescente.

# Merge: Esecuzione



# Merge: Esecuzione



# Merge()

---

**Merge**(**int**[] *A*, **int**[] *B*, **int** *start*, **int** *mid*, **int** *end*)

---

**int** *i* = *start*, *j* = *mid* + 1, *k* = *start***while** *i* ≤ *mid* **and** *j* ≤ *end* **do**    **if** *A*[*i*] ≤ *A*[*j*] **then**        | *B*[*k*] = *A*[*i*]        | *i* = *i* + 1    **else**        | *B*[*k*] = *A*[*j*]        | *j* = *j* + 1    | *k* = *k* + 1

% Prende dalla porzione sinistra

% Prende dalla porzione destra

**while** *i* ≤ *mid* **do**    | *B*[*k*] = *A*[*i*]    | *i* = *i* + 1; *k* = *k* + 1

% Copia ciò che resta a sinistra

**for** *t* = *start* **to** *k* - 1 **do**    | *A*[*t*] = *B*[*t*]% Ricopia il risultato in *A*

# Costo computazionale

## Domanda

Qual è il costo computazionale di `Merge()`?  $\Rightarrow \Theta(n)$

# Merge Sort: programma completo

---

```
MergeSort(int[] A, int n)
```

---

```
int[] B = new int[0...n - 1]
```

% Vettore di appoggio

```
MergeSortRec(A, B, 0, n - 1)
```

```
MergeSortRec(int[] A, int[] B, int start, int end)
```

```
    if start < end then
```

```
        int mid = ⌊(start + end)/2⌋
```

% Divide

```
        MergeSortRec(A, B, start, mid)
```

% Metà sinistra

```
        MergeSortRec(A, B, mid + 1, end)
```

% Metà destra

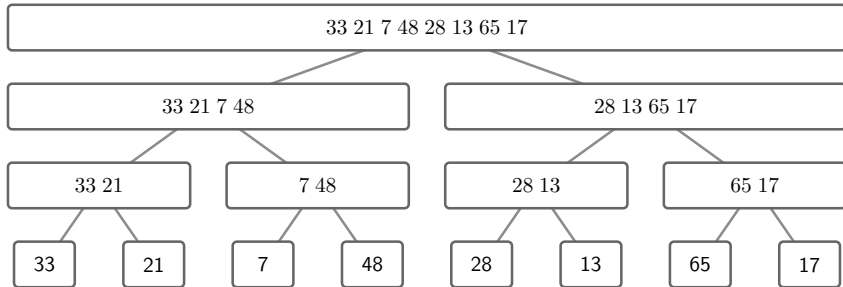
```
        Merge(A, B, start, mid, end)
```

% Combina

---

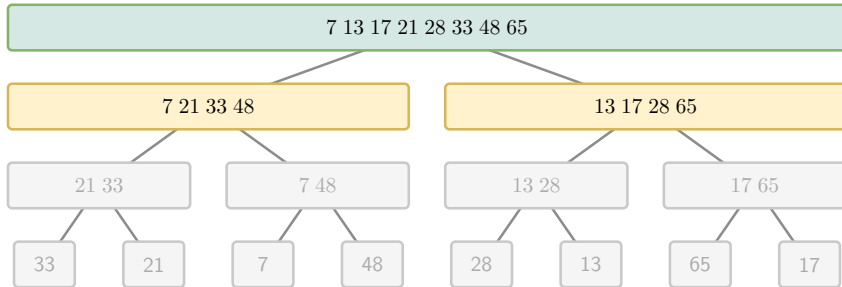
# MergeSort(): Esecuzione

## Suddivisione del problema



# MergeSort(): Esecuzione

## Sequenza ordinata



# Analisi di MergeSort()

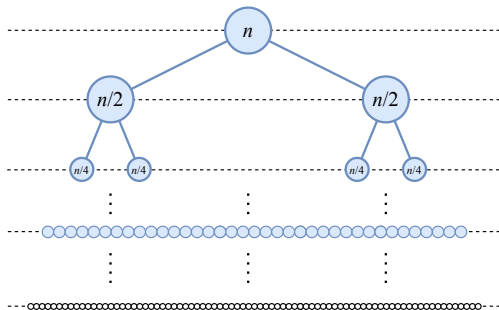
Un'assunzione semplificativa:

- $n = 2^k$ , quindi la profondità è esattamente  $k = \log n$ ;
- tutti i sottovettori hanno dimensione pari a una potenza di 2.

## Costo computazionale

$$T(n) = \begin{cases} d & n = 1 \\ 2T(n/2) + cn & n > 1 \end{cases}$$

# Analisi per livelli di Merge Sort



| Livello  | #            | Dim     | Costo                |
|----------|--------------|---------|----------------------|
| 0        | $2^0$        | $n/2^0$ | $c \cdot n$          |
| 1        | $2^1$        | $n/2^1$ | $c \cdot n$          |
| 2        | $2^2$        | $n/2^2$ | $c \cdot n$          |
| $i$      | $2^i$        | $n/2^i$ | $c \cdot n$          |
| $\log n$ | $2^{\log n}$ | 1       | $d \cdot 2^{\log n}$ |

$$T(n) = \sum_{i=0}^{\log n - 1} cn + dn = cn \log n + dn = \Theta(n \log n).$$

## Un po' di storia

- Il censimento americano del 1880 aveva richiesto otto anni per essere completato
- Quello del 1890 richiese sei mesi, grazie alla Hollerith Machine
- Fra il 1896 e il 1924, la Hollerith & Co ha cambiato diversi nomi. L'ultimo?  
**International Business Machines**
- Le Collating Machines (1936) fondevano due mazzi di schede perforate già ordinati in un unico mazzo ordinato
- Nel 1945-48, John von Neumann descrisse per la prima volta il MergeSort partendo dall'idea delle Collating Machines.



Hollerith Machine

# Limite inferiore per l'ordinamento per confronti

## Ordinamento per confronti

- Consideriamo algoritmi che ordinano **confrontando** coppie di elementi
- Supponiamo, per semplicità, che gli elementi siano distinti
- Esistono  $n!$  possibili ordinamenti dell'input
- Ogni confronto ha due esiti e corrisponde a un nodo di un **albero di decisione**
- Ogni foglia (terminazione) è associata a uno dei possibili ordinamenti

# Esempio di albero di decisione

**if**  $a < b$  **then**

**if**  $b < c$  **then**

**print**  $a, b, c$

**else**

**if**  $a < c$  **then**

**print**  $a, c, b$

**else**

**print**  $c, a, b$

**else**

**if**  $a < c$  **then**

**print**  $b, a, c$

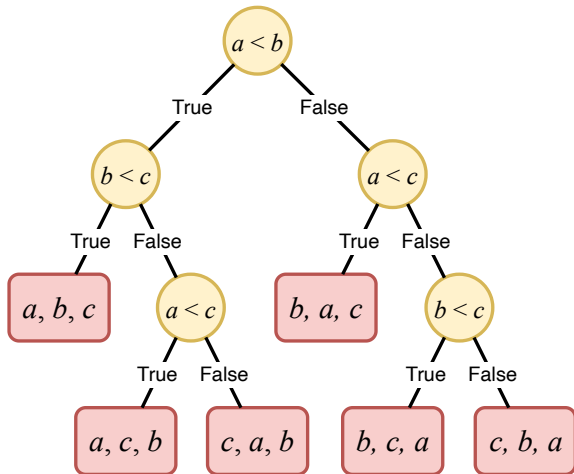
**else**

**if**  $b < c$  **then**

**print**  $b, c, a$

**else**

**print**  $c, b, a$



## Il limite inferiore è $\Omega(n \log n)$

### Dalle foglie all'altezza

Un albero binario di altezza  $h$  contiene al massimo  $2^h$  foglie. Per distinguere  $n!$  ordinamenti deve quindi valere

$$2^h \geq n! \quad \implies \quad h \geq \log_2(n!).$$

### Stimiamo $\log n!$

Assumiamo  $n$  pari. Almeno  $n/2$  fattori di  $n!$  sono maggiori o uguali a  $n/2$ :

$$\log_2(n!) \geq \log_2 \left( \frac{n}{2} \right)^{n/2} = \frac{n}{2} \log_2 \left( \frac{n}{2} \right) = \Omega(n \log n).$$

# Counting Sort cambia le ipotesi

## Assunzione più forte

Gli elementi di  $A$  sono interi compresi nell'intervallo  $[0 \dots k - 1]$ .

## Idea

- Usa  $B[0 \dots k - 1]$  per contare quante volte compare ciascun valore.
- Scorre  $B$  in ordine e riscrive in  $A$  le occorrenze contate.

Non confronta gli elementi fra loro: sfrutta direttamente il valore delle chiavi come indice.

# Counting Sort

---

```
countingSort(int[] A, int n, int k)
```

---

```
int[] B = new int[0...k-1] = {0}
```

% Contatori delle occorrenze

```
for j = 0 to n-1 do
```

```
    B[A[j]] += 1
```

% Conta A[j]

```
j = 0
```

```
for i = 0 to k-1 do
```

```
    while B[i] > 0 do
```

```
        A[j] = i
```

```
        j += 1
```

```
        B[i] -= 1
```

% Consuma un'occorrenza di i

---

## Costo

$$T(n, k) = \Theta(n + k).$$

## Perché non contraddice il limite inferiore?

$\Omega(n \log n)$  vale per l'ordinamento per confronti

Counting Sort non è basato su confronti e richiede chiavi intere in un intervallo noto: il modello e le ipotesi sono cambiati.

- Se  $k = \mathcal{O}(n)$ , Counting Sort è lineare.
- Se  $k = \Omega(n \log n)$ , non è asintoticamente più vantaggioso di Merge Sort.
- Se  $k = \Theta(2^n)$ , il costo  $\Theta(n + k)$  è addirittura superpolinomiale in  $n$ .

Un limite inferiore ha senso soltanto insieme alle operazioni consentite e alle informazioni disponibili sull'input.

# Sommario: algoritmi di ordinamento

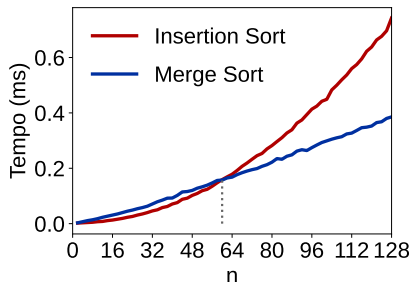
| Algoritmo      | Migliore           | Medio                    | Peggior                  | Stabile | Spazio                 |
|----------------|--------------------|--------------------------|--------------------------|---------|------------------------|
| Selection Sort | $\Theta(n^2)$      | $\Theta(n^2)$            | $\Theta(n^2)$            | No      | $\Theta(1)$            |
| Insertion Sort | $\Theta(n)$        | $\Theta(n^2)$            | $\Theta(n^2)$            | Sì      | $\Theta(1)$            |
| Merge Sort     | $\Theta(n \log n)$ | $\Theta(n \log n)$       | $\Theta(n \log n)$       | Sì      | $\Theta(n)$            |
| Counting Sort  | $\Theta(n + k)$    | $\Theta(n + k)$          | $\Theta(n + k)$          | No      | $\Theta(k)$            |
| Shell Sort     | $\Theta(n)$        | $\mathcal{O}(n\sqrt{n})$ | $\mathcal{O}(n\sqrt{n})$ | No      | $\Theta(1)$            |
| Quick Sort     | $\Theta(n \log n)$ | $\Theta(n \log n)$       | $\Theta(n^2)$            | No      | $\Theta(\log n)$ medio |
| Heap Sort      | $\Theta(n \log n)$ | $\Theta(n \log n)$       | $\Theta(n \log n)$       | No      | $\Theta(1)$            |

## Stabilità

Un algoritmo è stabile se gli elementi con la stessa chiave mantengono l'ordine relativo che avevano nell'input.

# Ordine asintotico e costanti

- Per input grandi, l'ordine di crescita domina i fattori costanti.
- Per input piccoli contano anche chiamate, allocazioni, memoria e architettura.
- La soglia di incrocio dipende da implementazione e macchina.



Nell'esperimento mostrato, un'implementazione di Merge Sort che utilizza Insertion Sort per  $n \leq 64$  è più efficiente di quella che utilizza  $n = 1$  come caso base.

# Moltiplicazione: evoluzione storica

| Algoritmo                    | Anno | Complessità in bit                                 |
|------------------------------|------|----------------------------------------------------|
| Moltiplicazione scolastica   | —    | $\Theta(n^2)$                                      |
| Karatsuba–Ofman (Toom-2)     | 1962 | $\Theta(n^{\log_2 3})$ , $\log_2 3 \simeq 1.585$   |
| Toom–Cook (Toom-3)           | 1963 | $\Theta(n^{\log_3 5})$ , $\log_3 5 \simeq 1.465$   |
| Toom–Cook (Toom-4)           | 2006 | $\Theta(n^{\log_4 7})$ , $\log_4 7 \simeq 1.404$   |
| Toom–Cook (Toom-6.5)         | 2009 | $\Theta(n^{\log_6 12})$ , $\log_6 12 \simeq 1.387$ |
| Toom–Cook (Toom-8.5)         | 2009 | $\Theta(n^{\log_8 16}) = \Theta(n^{4/3})$          |
| Schönhage–Strassen (FFT)     | 1971 | $\mathcal{O}(n \log n \log \log n)$                |
| Fürer                        | 2007 | $\mathcal{O}(n \log n 2^{\mathcal{O}(\log^* n)})$  |
| Harvey–van der Hoeven–Lecerf | 2014 | $\mathcal{O}(n \log n 8^{\log^* n})$               |
| Harvey–van der Hoeven        | 2018 | $\mathcal{O}(n \log n 4^{\log^* n})$               |
| Harvey–van der Hoeven        | 2021 | $\mathcal{O}(n \log n)$                            |

## Moltiplicazione: considerazioni finali

### Che cosa sappiamo del problema?

La limitazione superiore è  $\mathcal{O}(n \log n)$ ; nel modello generale, non è noto un limite inferiore migliore di  $\Omega(n)$ . L'ottimalità resta quindi aperta.

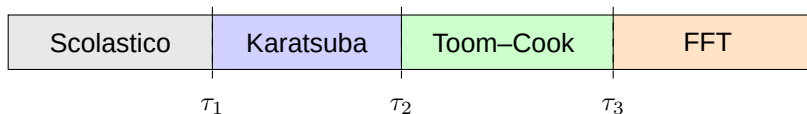
### Dalla teoria alla pratica

Un esponente più piccolo promette una crescita migliore, ma spesso introduce operazioni aggiuntive e costanti più elevate.

# GMP combina algoritmi diversi

## GNU Multiple Precision Arithmetic Library

Il [manuale di GMP](#) descrive una successione di algoritmi scelta in funzione della dimensione degli operandi.



- Le soglie dipendono da architettura, compilatore e implementazione.
- Vengono determinate sperimentalmente, non dalla sola notazione asintotica.